Name:Abhishek yadav. USN No.:CS23071 Aim:To deploy a trained machine learning model using a suitable framework or platform and test it with real-time or unseen data.

```
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
path = "/content/drive/MyDrive/DATASET_DAILY_WEATHER/daily_weather.csv"
df=pd.read_csv(path)
df.head(5)
```

	number	air_pressure_9am	air_temp_9am	avg_wind_direction_9am	avg_wind_speed_9am	max_wind_direction_9am	max_wind_speed_9a
0	0	918.060000	74.822000	271.100000	2.080354	295.400000	2.86328
1	1	917.347688	71.403843	101.935179	2.443009	140.471548	3.53332
2	2	923.040000	60.638000	51.000000	17.067852	63.700000	22.10096
3	3	920.502751	70.138895	198.832133	4.337363	211.203341	5.19004
4	4	921.160000	44.294000	277.800000	1.856660	136.500000	2.86328

Next steps: Generate code with df New interactive sheet

```
df.shape
```

```
(1095, 11)
```

```
df.columns
```

```
Index(['number', 'air_pressure_9am', 'air_temp_9am', 'avg_wind_direction_9am',
       'avg_wind_speed_9am', 'max_wind_direction_9am', 'max_wind_speed_9am',
       'rain_accumulation_9am', 'rain_duration_9am', 'relative_humidity_9am',
       'relative_humidity_3pm'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.True_
```

```
df.isnull().sum()
```

	0
number	0
air_pressure_9am	3
air_temp_9am	5
avg_wind_direction_9am	4
avg_wind_speed_9am	3
max_wind_direction_9am	3
max_wind_speed_9am	4
rain_accumulation_9am	6
rain_duration_9am	3
relative_humidity_9am	0
relative_humidity_3pm	0

dtype: int64

```
df[df.isnull().any(axis = 1)]
```

	number	air_pressure_9am	air_temp_9am	avg_wind_direction_9am	avg_wind_speed_9am	max_wind_direction_9am	max_wind_speed
16	16	917.890000	NaN	169.200000	2.192201	196.800000	2.93
111	111	915.290000	58.820000	182.600000	15.613841	189.000000	
177	177	915.900000	NaN	183.300000	4.719943	189.900000	5.34
262	262	923.596607	58.380598	47.737753	10.636273	67.145843	13.67
277	277	920.480000	62.600000	194.400000	2.751436	NaN	3.86
334	334	916.230000	75.740000	149.100000	2.751436	187.500000	4.18
358	358	917.440000	58.514000	55.100000	10.021491	NaN	12.70
361	361	920.444946	65.801845	49.823346	21.520177	61.886944	25.54
381	381	918.480000	66.542000	90.900000	3.467257	89.400000	4.40
409	409	NaN	67.853833	65.880616	4.328594	78.570923	5.21
517	517	920.570000	53.600000	100.100000	4.697574	NaN	6.28
519	519	916.250000	55.670000	176.400000	6.666081	188.200000	
546	546	NaN	42.746000	251.100000	12.929513	274.400000	17.60
620	620	921.200000	56.786000	192.300000	9.551734	201.400000	11.00
625	625	912.400000	50.774000	171.600000	NaN	181.400000	4.83
656	656	920.830000	66.344000	NaN	15.457255	189.400000	16.48
670	670	910.920000	48.362000	156.500000	NaN	177.500000	16.12
672	672	922.448945	72.863773	NaN	3.682370	214.196160	4.84
705	705	911.900000	59.072000	199.800000	1.275056	239.500000	1.83
731	731	922.970166	51.391847	33.810942	NaN	59.290089	11.11
737	737	917.895130	76.804690	104.771020	1.632705	97.178763	
788	788	917.923442	73.249717	42.101739	4.132698	64.284969	5.34
840	840	918.043767	NaN	181.774042	0.964376	185.618601	1.57
848	848	915.250000	37.562000	246.500000	11.587349	258.700000	
861	861	919.065408	NaN	172.303728	2.639600	193.058141	3.32
869	869	NaN	45.104000	259.000000	3.265932	275.000000	4.02
998	998	914.140000	71.240000	NaN	1.722444	232.900000	2.32
1031	1031	922.669195	NaN	47.946284	7.969686	65.770066	10.26
1035	1035	919.670000	77.576000	171.800000	6.554234	191.000000	8.16
1063	1063	917.300185	65.790001	NaN	1.879553	222.498226	2.69
1066	1066	919.564869	73.726732	68.704694	3.551777	102.571616	4.86

```
del df['number']
df.columns
```

```
Index(['air_pressure_9am', 'air_temp_9am', 'avg_wind_direction_9am',
       'avg_wind_speed_9am', 'max_wind_direction_9am', 'max_wind_speed_9am',
       'rain_accumulation_9am', 'rain_duration_9am', 'relative_humidity_9am',
       'relative_humidity_3pm'],
      dtype='object')
```

```
df.shape
```

```
(1095, 10)
```

```
before_rows = df.shape[0]
df = df.dropna()
after_rows = df.shape[0]

print("Before rows:", before_rows)
```

```
print("After rows:", after_rows)
print("The number of dropped rows are:", before_rows - after_rows)
```

```
Before rows: 1095
After rows: 1064
The number of dropped rows are: 31
```

```
df.shape
```

```
(1064, 10)
```

```
clean_data = df.copy()
clean_data['high_humidity_label'] = (clean_data['relative_humidity_3pm'] > 24.99)*1
print(clean_data['high_humidity_label'])
```

```
0       1
1       0
2       0
3       0
4       1
       ..
1090    1
1091    1
1092    1
1093    1
1094    0
Name: high_humidity_label, Length: 1064, dtype: int64
```

```
clean_data.columns
```

```
Index(['air_pressure_9am', 'air_temp_9am', 'avg_wind_direction_9am',
       'avg_wind_speed_9am', 'max_wind_direction_9am', 'max_wind_speed_9am',
       'rain_accumulation_9am', 'rain_duration_9am', 'relative_humidity_9am',
       'relative_humidity_3pm', 'high_humidity_label'],
      dtype='object')
```

```
del clean_data['relative_humidity_3pm']
del clean_data['relative_humidity_9am']
clean_data.columns
```

```
Index(['air_pressure_9am', 'air_temp_9am', 'avg_wind_direction_9am',
       'avg_wind_speed_9am', 'max_wind_direction_9am', 'max_wind_speed_9am',
       'rain_accumulation_9am', 'rain_duration_9am', 'high_humidity_label'],
      dtype='object')
```

```
from sklearn.model_selection import train_test_split
X = clean_data.drop('high_humidity_label', axis=1)
y = clean_data['high_humidity_label']
X_train, X_test, y_train, y_test = train_test_split(X,y, train_size = 0.7, test_size= 0.3, random_state = 42)
```

```
X_train
```

	air_pressure_9am	air_temp_9am	avg_wind_direction_9am	avg_wind_speed_9am	max_wind_direction_9am	max_wind_speed_9am	ra
5	915.30000	78.404000	182.800000	9.932014	189.000000	10.983375	
315	917.09000	63.266000	175.500000	6.733189	186.300000	7.963506	
918	915.07000	58.046000	186.200000	7.695074	195.600000	8.992499	
105	917.40000	73.868000	181.700000	4.138339	189.800000	4.764682	
98	916.52000	51.638000	190.800000	7.404271	198.000000	8.388525	
...	...	...	...	...	...	...	
336	912.89000	45.320000	213.400000	21.250930	220.500000	23.577348	
476	917.18000	62.834000	186.300000	3.221194	197.700000	4.026492	
123	920.18000	62.744000	32.600000	3.019869	51.300000	3.825167	
1075	922.85811	64.989361	63.483047	10.261187	73.170504	11.949206	
886	918.70000	69.422000	161.100000	0.782929	193.600000	1.185578	

744 rows × 8 columns

Next steps: Generate code with X_train New interactive sheet

```
X_test
```

	air_pressure_9am	air_temp_9am	avg_wind_direction_9am	avg_wind_speed_9am	max_wind_direction_9am	max_wind_speed_9am	ra
32	918.700000	76.226000	184.600000	4.093600	194.900000	4.742313	
1084	917.130000	80.240000	183.300000	1.632966	38.500000	2.259309	
423	921.090000	70.124000	182.800000	2.908022	204.500000	3.534365	
767	915.220000	73.292000	148.300000	1.789552	198.500000	2.908022	
818	921.400000	63.050000	223.200000	1.319795	185.000000	1.879030	
...	...	...	...	...	...	...	
503	919.738161	67.432744	147.470882	1.157775	129.981819	1.899000	
585	916.250000	79.754000	48.600000	1.073731	99.800000	1.588227	
389	916.500000	74.588000	101.200000	1.834291	132.400000	2.594850	
715	923.600000	65.030000	62.300000	7.091100	75.100000	8.410894	
352	925.280000	66.470000	49.400000	11.229439	77.900000	15.345408	

320 rows × 8 columns

Next steps: Generate code with X_test New interactive sheet

```
y_train
```

	high_humidity_label
5	1
315	1
918	1
105	1
98	1
...	...
336	1
476	1
123	1
1075	0
886	0

744 rows × 1 columns

dtype: int64

```
y_test
```

	high_humidity_label
32	1
1084	1
423	1
767	1
818	1
...	...
503	0
585	1
389	1
715	1
352	0

320 rows × 1 columns

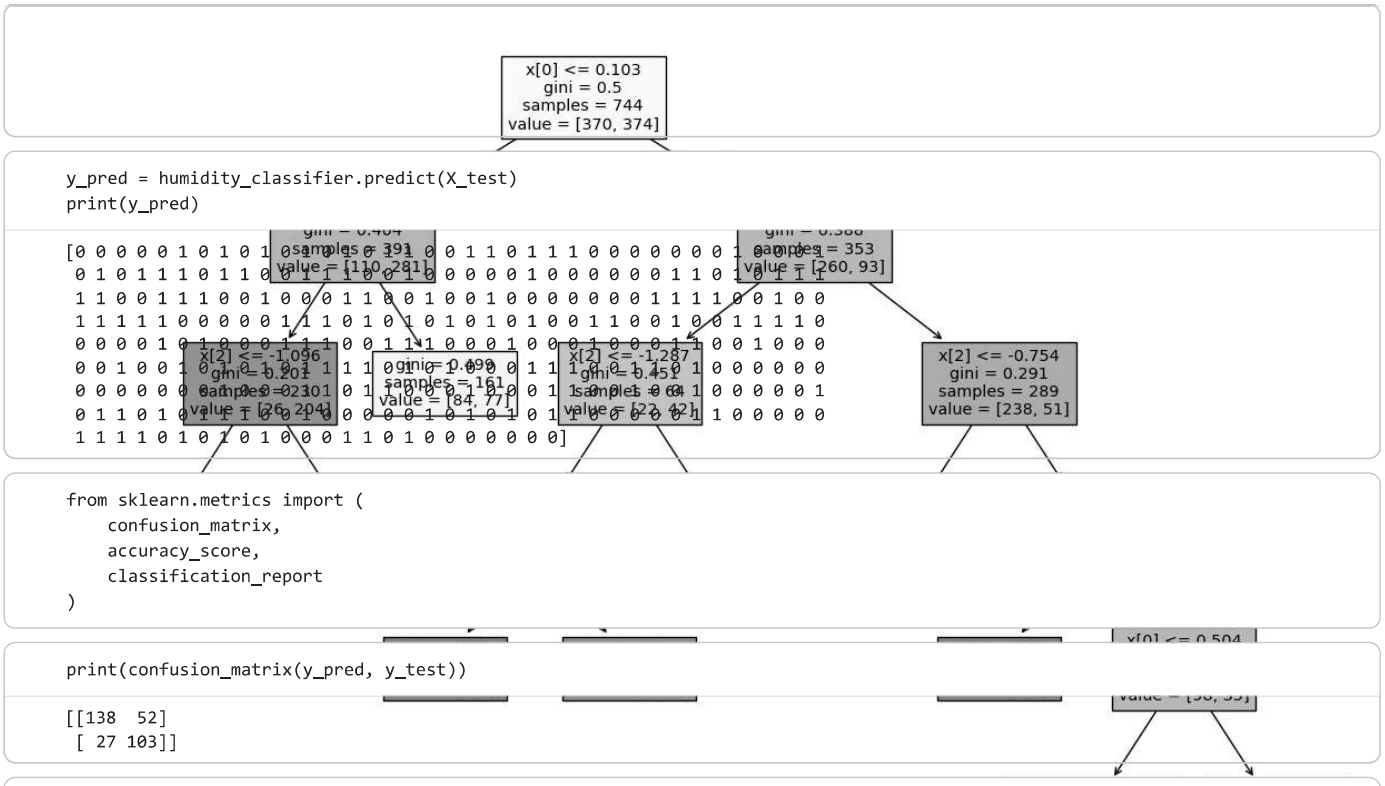
dtype: int64

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
humidity_classifier = DecisionTreeClassifier (max_leaf_nodes=10, random_state=0)
humidity_classifier.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_leaf_nodes=10, random_state=0)
```

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
plt.figure(figsize=(15,10))
tree = plot_tree(humidity_classifier, filled=True)
```




```
y_pred = humidity_classifier.predict(X_test)
print(y_pred)
```

```
[0 0 0 0 0 1 0 1 0 1 0 1 1 0 1 1 0 0 1 1 0 1 1 1 0 0 0 0 0 0 1 0 0 0 1
 0 1 0 1 1 1 0 1 1 0 0 1 1 0 0 1 0 0 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 1 1
 1 1 0 0 1 1 1 0 0 1 0 0 0 1 1 0 0 1 0 0 1 0 0 0 0 0 0 1 1 1 1 0 0 1 0 0
 1 1 1 1 1 0 0 0 0 0 1 1 1 0 1 0 1 0 1 0 1 0 1 0 0 1 1 0 0 1 0 0 1 1 1 1 0
 0 0 0 0 1 0 1 0 0 0 1 1 1 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 0 1 1 0 0 1 0 0 0
 0 0 1 0 0 1 0 1 0 1 1 1 0 0 1 1 1 0 0 1 1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 0 1 0 0 1 0 0 1 0 0 1 0 0 0 0 0 0 1
 0 1 1 0 1 0 1 1 1 0 1 0 0 0 0 1 0 1 0 1 0 1 1 0 0 0 0 1 1 0 0 0 0 0
 1 1 1 1 0 1 0 1 0 1 0 0 1 1 0 1 0 0 0 0 0 0 0]
```

```
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    classification_report
)
```

```
print(confusion_matrix(y_pred, y_test))
```

```
[[138  52]
 [ 27 103]]
```

```
print(classification_report (y_pred, y_test))
```

```
              precision    recall  f1-score   support

           0       0.84      0.73      0.78       190
           1       0.66      0.79      0.72       130

    accuracy                           0.75       320
   macro avg       0.75      0.76      0.75       320
weighted avg       0.77      0.75      0.76       320
```

```
import pickle
with open('model.pkl', 'wb') as f:
    pickle.dump(humidity_classifier, f)
with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

print("Model and Scaler Saved!")
```

```
Model and Scaler Saved!
```